EE345 Mod4-Lec2: Error Analysis and Model Selection

[VMLS, 14]

Prediction Errors

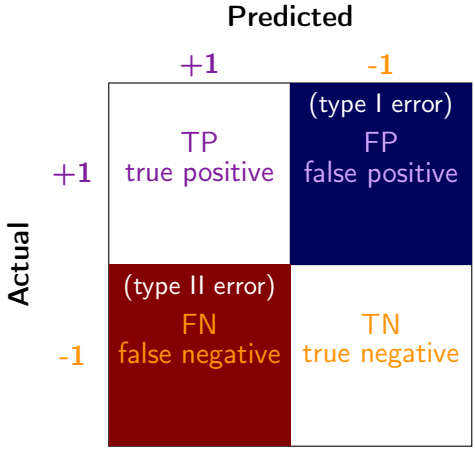
For a given data point (x, y) with predicted outcome $\hat{y} = f(x)$, there are four possibilities:

1. True Positive (TP): $y = +1$ and $\hat{y} = +1$ [correct]
2. True Negative (TN): $y = -1$ and $\hat{y} = -1$ [correct]
3. False Positive (FP): $y = -1$ and $\hat{y} = +1$ [incorrect , type I error]
4. False Negative (FN): $y = +1$ and $\hat{y} = -1$ [incorrect, type II error]

Confusion Matrix

	prediction		
$y \backslash \hat{y}$	$+1$	-1	total
$+1$	$N_{\mathtt{tp}}$	$N_{\mathtt{fn}}$	$N_{\mathtt{p}}$
-1	$N_{\mathtt{fp}}$	$N_{\mathtt{tn}}$	$N_{\mathtt{n}}$
all	$N_{\mathtt{tp}} + N_{\mathtt{fp}}$	$N_{\mathtt{fn}} + N_{\mathtt{tp}}$	N

- *Good classifier*:
  - small (near zero) error rate and false positive rate,
  - high (near one) true positive rate, true negative rate, and precision.
  - well calibrated predictions.



Counting Errors

Consider data set $(x_1, \ldots, x_N)$, $(y_1, \ldots, y_N)$ and model f with $N = N_{\mathrm{p}} + N_{\mathrm{n}}$ outcomes.

- **Error rate**:
- **False positive rate** (false alarm):
- **True negative rate** (specificity):

Actual \ Predicted	+1	-1
+1	N_{tp} # true positives $y = +1$ and $\hat{y} = +1$	N_{fp} # false positives $y = -1$ and $\hat{y} = +1$
-1	N_{fn} # false negatives $y = +1$ and $\hat{y} = -1$	N_{tn} # true negatives $y = -1$ and $\hat{y} = -1$

More Error Rates & Metrics

- $N_{\mathtt{p}} = N_{\mathtt{tp}} + N_{\mathtt{fn}}$: # of outcomes with a true positive label $y = +1$
- $N_{\mathtt{n}} = N_{\mathtt{fp}} + N_{\mathtt{tn}}$: # of outcomes with a true negative label $y = -1$
- **True positive rate** (sensitivity/**recall**):

- **Precision**:

- **F1 score**: harmonic mean of precision and recall

F1 Score: Balancing Precision and Recall

Why Do We Need It?

- Precision and recall often move in *opposite* directions.
- F1 gives a single number that increases only when *both* are high.
- Uses a harmonic mean to penalize imbalance: precision $\gg$ recall $\implies$ F1 is small.

When to Use F1?

- Positive class is rare (imbalanced datasets).
- Care about detecting positives, but also want predictions to be reliable.
- Do *not* care about TNs (true negatives) as much.
- Often used in:
  - information retrieval,
  - medical diagnosis,
  - fraud/anomaly detection.

Precision, Recall, False Alarm, and ROC

When to Care About Which?

- **Precision** (How trustworthy are positive predictions?)
  - Important when false positives are costly;
    Example: flagging emails as spam, fraud alerts.
- **Recall / TPR** (How many real positives did we catch?)
  - Important when missing a positive is costly;
    Example: disease screening, safety-critical detection.
- **False Alarm Rate / FPR** (How often do we cry wolf?)
  - Important when too many negatives are incorrectly flagged;
    Example: security systems, anomaly detection.

Iris Data Set Example: Confusion Matrix

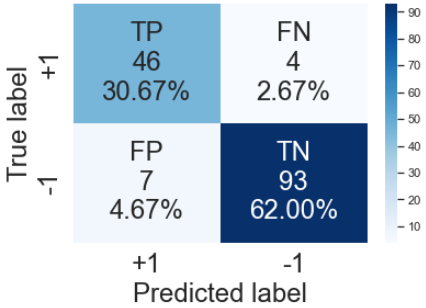
- Precision:

$$\frac{N_{\mathtt{tp}}}{N_{\mathtt{tp}} + N_{\mathtt{fp}}}$$

- Accuracy:

$$\frac{N_{\mathtt{tp}} + N_{\mathtt{tn}}}{N}$$

- F1-score is the harmonoic mean of precision (P) and recall (R): $\frac{2PR}{P+R}$
  - recall: $N_{\mathtt{tp}}/(N_{\mathtt{tp}} + N_{\mathtt{fn}})$
  - want it near 1



Accuracy=0.927; Precision=0.959
Recall=0.930; F1 Score=0.944

ROC and Precision–Recall Curves

Two Curves: ROC and Precision–Recall

- ROC curve shows **Recall (TPR)** $N_{\mathtt{tp}}/(N_{\mathtt{tp}} + N_{\mathtt{fn}})$ vs **FPR** $N_{\mathtt{fp}}/(N_{\mathtt{fp}} + N_{\mathtt{tn}})$ across thresholds.
- Precision–Recall curve shows how the reliability of your positive predictions changes as recall increases (ability to catch positives).

Reciever Operating Characteristic (ROC) Curves

Many classifiers output a **score** or **probability**:

$$s_i = p(y_i = 1 \mid x_i).$$

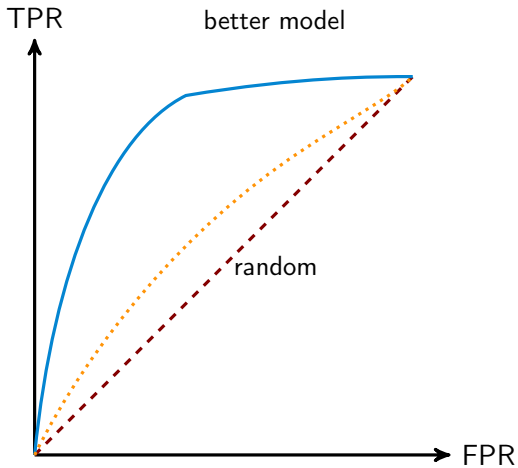
To turn scores into predictions, choose a **threshold** t and predict:

Receiver Operating Characteristic (ROC) curve:

Useful when:

- classes are imbalanced,
- the cost of FP vs FN differs (cancer prediction: FP = false alarm, FN = missing cancer),
- we want threshold-independent comparison.

Example ROC Curves

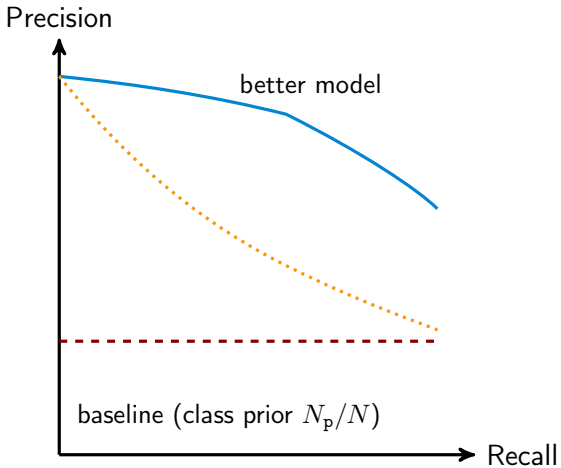


For each threshold t:

$$\text{TPR}(t) = \frac{N_{\texttt{tp}}(t)}{N_{\texttt{tp}}(t) + N_{\texttt{fn}}(t)}$$

$$\text{FPR}(t) = \frac{N_{\texttt{fp}}(t)}{N_{\texttt{fp}}(t) + N_{\texttt{tn}}(t)}$$

Example Precision–Recall Curves



$$\text{Precision} = \frac{N_{\mathtt{tp}}(t)}{N_{\mathtt{tp}}(t) + N_{\mathtt{fp}}(t)},$$

$$\text{Recall (TPR)} = \frac{N_{\mathtt{tp}}(t)}{N_{\mathtt{tp}}(t) + N_{\mathtt{fn}}(t)},$$

AUC: A Threshold-Independent Score

AUC = area under the ROC curve.
Interpretation:

- AUC = 1: perfect classifier.
- AUC = 0.5: random guessing.
- AUC = $\Pr(\text{random positive} > \text{random negative in score})$ (random guessing).

Why useful?

- Does not depend on any particular threshold.
- Robust to class imbalance.
- Measures how well scores *rank* positives above negatives.

In practice, ROC curves + AUC are standard for evaluating classifiers with probabilistic outputs (e.g. logistic regression).

Probability Calibration

What Is Probability Calibration?

A classifier outputs predicted probabilities $\hat{p}(x)$.
Calibration means:

$$\Pr(y = 1 \mid \hat{p}(X) \approx p) \approx p \quad \text{for all } p \in [0, 1].$$

Examples:

- Among all predictions near $\hat{p} = 0.2$, about 20% should actually be positive.
- Among all predictions near $\hat{p} = 0.8$, about 80% should be positive.

Well-calibrated models:

- logistic regression (usually),
- naive Bayes (poor),
- random forests (medium),
- deep nets (overconfident, often poorly calibrated).

Calibration is **separate** from accuracy.

How to Compute a Calibration Curve (Reliability Diagram)

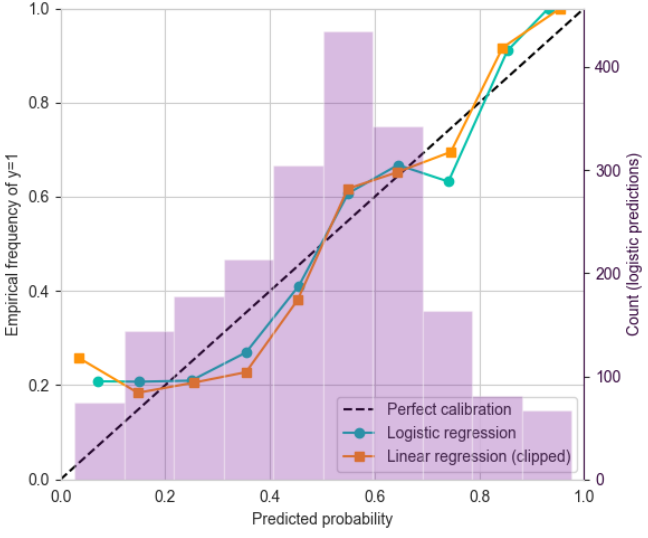
Given predicted probabilities p_i and true labels y_i:

1. Split $[0, 1]$ into bins, e.g.

$$[0, 0.1), [0.1, 0.2), \ldots, [0.9, 1.0].$$

2. For each bin B:
   - $\hat{p}_B$ = average predicted probability in the bin,
   - $\hat{y}_B$ = empirical fraction of positives in the bin.
3. Plot points $(\hat{p}_B, \hat{y}_B)$.

Calibration Curve Interpretation



Interpretation:

- **Perfect calibration:** points lie on the diagonal $y = x$.
- Above diagonal $\Longrightarrow$ underconfident.
- Below diagonal $\Longrightarrow$ overconfident.

Why Does Calibration Matter?

Decision-making depends on probabilities.
Examples:

- Medical diagnosis: $\hat{p} = 0.9$ should *really* mean 90% risk.
- Fraud detection: thresholding depends on calibrated scores.
- Multi-class classification: softmax outputs are often overconfident.

A classifier can have:

- high accuracy but poor calibration,
- good calibration but mediocre accuracy.

ROC curves measure ranking quality. Calibration measures probability quality.
Both are important, but different.

ROC Curves vs Calibration: When to Use Which?

Question	**Use ROC / AUC**	**Use Calibration Curve**
How well does the model *rank* positives above negatives?	Y: ROC/AUC measure ranking quality	
Predicted probabilities *meaningful*?		Y: calibration measures probability accuracy
Class imbalance (rare positives)?	ROC/AUC are robust (insensitive) to imbalance	Calibration still works, but bins may need merging
Threshold selection (varying decision threshold)?	ROC ideal: shows tradeoff across all thresholds	Not designed for threshold choice

ROC Curves vs Calibration: When to Use Which?

Question	Use ROC / AUC	Use Calibration Curve
Decision-making under asymmetric costs (e.g. FP vs FN)?	ROC helps visualize tradeoffs	Calibration helps if decisions use probability estimates
Model comparison when only ranking matters (e.g. search, ranking tasks)?	ROC/AUC preferred	
Model comparison when downstream tasks use predicted probabilities (e.g. risk prediction)?		Calibration is essential
Neural networks (often overconfident)?	AUC may still look good	Calibration will reveal overconfidence

Summary: ROC curves evaluate *ranking*. Calibration curves evaluate *probability quality*. Both measure different, and complementary, aspects of model performance.

Cross Validation for Model Selection

Why Cross Validation?

Given data $\{(x_i, y_i)\}_{i=1}^{n}$, we want to estimate how well a model generalizes.
Problem: If we train and evaluate on the same data:

- training error is overly optimistic,
- overfitting looks good,
- we cannot compare models fairly.

Solution: Split the data into

- a set used for **training**, and another set used only for **validation**.

But a single split may be:

- unlucky,
- unstable (high variance),
- too small for training or validation.

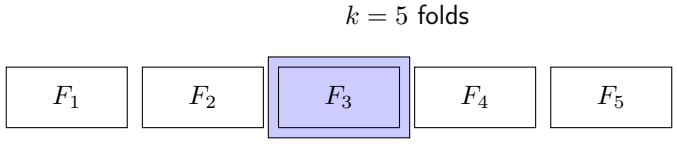
Cross validation averages over many splits to stabilize the estimate.

k-Fold Cross Validation Steps

k-**fold cross validation** works as follows:

Other CV Metrics

Visualization of k-Fold Cross Validation



Training = F_1, F_2, F_4, F_5

In each round, we rotate the validation fold and retrain the model. After k rounds, we average the k validation errors.

Cross Validation for Model Selection

Suppose we want to choose amongst the following hyperparameters:

For each candidate model $\mathcal{M}(\lambda, p, \kappa, \mathcal{C})$:

Aside: Kernel Bandwidth (i.e. parameterization of a kernel)

In kernel methods (kernel regression, density estimation, RBF kernels), we use a kernel $K_\kappa(x, x_i)$ to measure the "similarity" between points. The parameter κ = **bandwidth** controls how wide or narrow the kernel is.

Example (Gaussian kernel):

$$K_\kappa(x, x_i) = \exp\left(-\frac{\|x - x_i\|^2}{2\kappa^2}\right).$$

Interpretation of bandwidth:

- **Small** κ: kernels are narrow $\implies$ very local, high flexibility, can overfit
- **Large** κ: kernels are wide $\implies$ heavy smoothing, risk of underfitting

Cross Validation for Model Selection

Choose the model with the lowest CV error:

Why Not Just One Train/Validation Split?

A single validation split has problems:

- **High variance**: the estimate depends on one partition.
- **Wasted data**: validation set not used for training.
- **Unfair to models**: one split may favor some models.
- **Unstable hyperparameters**.

Cross validation:

- uses *all* data for training and validation (on rotation),
- yields a more reliable estimate of generalization,
- reduces randomness from dataset splits,
- improves hyperparameter selection.

Choosing k and Practical Considerations

Common choices:

- $k = 5$ or 10: good bias/variance tradeoff.
- $k = n$: **Leave-One-Out CV** (LOOCV) — high variance, costly.
- $k = 2$: simple split (train/test).

Practical notes:

- Shuffle data before splitting.
- Stratify folds for classification:

  class proportions preserved in each fold.

- Use the **same folds** for all models being compared.
- After selecting hyperparameters, retrain on all data.

Cross validation is the gold standard for model selection in classical ML.

Extra Slides: Maximum Likelihood Estimation

From Probabilistic Modeling to Maximum Likelihood

In logistic regression, we model the conditional distribution:

$$p(y_i = 1 \mid x_i; \theta) = \sigma(\theta^\top x_i), \qquad p(y_i = 0 \mid x_i; \theta) = 1 - \sigma(\theta^\top x_i),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$.
Assumption: Each label y_i is generated independently from a **Bernoulli** distribution:

$$y_i \sim \text{Bernoulli}\big(\sigma(\theta^\top x_i)\big).$$

Therefore, the likelihood is:

$$L(\theta) = \prod_{i=1}^{n} \Big[\sigma(\theta^\top x_i)\Big]^{y_i} \Big[1 - \sigma(\theta^\top x_i)\Big]^{1-y_i}.$$

This is the probability of observing the labels $\{y_i\}$ under the model.

What Is a Likelihood?

Suppose we have:

- data (observations) D,
- a parametric model $p(D \mid \theta)$.

Definition (Likelihood). The *likelihood function* is

$$L(\theta) := p(D \mid \theta),$$

viewed as a function of the parameter θ with the data D fixed.

- Probability: a function of the **observed outcomes**.
- Likelihood: the **same quantity**, but viewed as a function of **parameters**.

Goal in Maximum Likelihood: Find parameter θ that makes the observed data most probable:

$$\hat{\theta}_{\mathsf{MLE}} = \arg\max_{\theta} L(\theta).$$

Likelihood Example: A Biased Coin

Suppose we flip a coin n times and see outcomes $y_1, \ldots, y_n \in \{0, 1\}$.
Model:

$$y_i \sim \text{Bernoulli}(p), \qquad p = \Pr(\text{Heads}).$$

Probability of the whole dataset:

$$p(y_1, \ldots, y_n \mid p) = \prod_{i=1}^{n} p^{y_i}(1-p)^{1-y_i}.$$

As a function of p, this is the **likelihood**:

$$L(p) = p^{\sum_i y_i}(1-p)^{n-\sum_i y_i}.$$

Interpretation: This tells us how plausible different values of p are, *given the data we observed*.

Likelihood in Supervised Learning

In supervised learning, we observe pairs (x_i, y_i).
To define a likelihood, we must:

1. Choose a model for $p(y \mid x, \theta)$;
2. Assume the training examples are **independent**.

Then the joint probability of all labels given inputs $X = \{x_i\}$ and parameters θ is:

$$p(y_1, \ldots, y_n \mid x_1, \ldots, x_n, \theta) = \prod_{i=1}^{n} p(y_i \mid x_i, \theta).$$

This joint probability, treated as a function of θ, is the **likelihood**:

$$L(\theta) = \prod_{i=1}^{n} p(y_i \mid x_i, \theta).$$

Maximum likelihood picks θ making the observed labels most probable.

Likelihood for Logistic Regression

Model the conditional distribution:

$$p(y_i = 1 \mid x_i, \theta) = \sigma(\theta^\top x_i), \qquad p(y_i = 0 \mid x_i, \theta) = 1 - \sigma(\theta^\top x_i).$$

Assumption: The labels $y_1, \ldots, y_n$ are conditionally independent given the inputs.
Joint probability of the dataset:

$$p(y_1, \ldots, y_n \mid x_1, \ldots, x_n, \theta) = \prod_{i=1}^{n} \Big[\sigma(\theta^\top x_i)\Big]^{y_i} \Big[1 - \sigma(\theta^\top x_i)\Big]^{1-y_i}.$$

This is the likelihood:

$$L(\theta) = \prod_{i=1}^{n} \Big[\sigma(\theta^\top x_i)\Big]^{y_i} \Big[1 - \sigma(\theta^\top x_i)\Big]^{1-y_i}.$$

Maximizing this is equivalent to minimizing the logistic loss.